

PETGUARD PRO

Smart IoT Pet Monitoring & Safety System

Design Manual

Repository: [cepdnack/e21-3yp-PetGuard-Pro](#)

Group:

E/21/106 — Dissanayaka D.M.M.B.

E/21/137 — Fernando W.W.C.C.J.

E/21/350 — Samarawickrama G.B.B.P.

E/21/428 — Weerasooriya S.C.

Supervisors:

Ms. Yasodha Vimukthi

Dr. Isuru Nawinne

Mr. Thilina Gunarathne

Department of Computer Engineering, University of Peradeniya

Table of Content

1. Introduction
 - 1.1 Project Overview
 - 1.2 Objective and Scope
 - 1.3 Solution Architecture
 - 1.4 Data Flow
 - 1.5 Security
2. Software Infrastructure
 - 2.1 Frontend
 - 2.1.1 Key Features
 - 2.1.2 Technology Stack
 - 2.1.3 UI Design
 - 2.2 Backend
 - 2.2.1 Introduction
 - 2.2.2 Key Features
 - 2.2.3 Technology Stack
 - 2.2.4 Modular Architecture and Data Flow
 - 2.2.5 Firebase REST Interface (Firmware & Cloud)
3. Cloud Deployment Architecture and Procedure
 - 3.1 Architecture Overview
 - 3.1.1 Multi-Tier Architecture Design
 - 3.1.2 Cloud Infrastructure Components
 - 3.2 Firebase Project Setup
 - 3.2.1 Firebase Account & Project Configuration
 - 3.2.2 Realtime Database & Firestore Creation
 - 3.2.3 Hosting Setup for Admin Dashboard
 - 3.3 Environment & Security Configuration
 - 3.3.1 Environment Variables Setup
 - 3.3.2 Security Rules Configuration
 - 3.3.3 Authentication Configuration
 - 3.4 Database, Frontend and Backend Deployment
 - 3.4.1 Realtime Database & Firestore Setup
 - 3.4.2 Authentication & Cloud Messaging Configuration
 - 3.4.3 Admin Dashboard Deployment Process
 - 3.4.4 Mobile App Deployment Process
 - 3.5 Service Management
 - 3.5.1 Application Monitoring
 - 3.5.2 Logging Configuration
 - 3.5.3 Offline Resilience (Flash Buffering)
 - 3.6 Client Configuration
 - 3.6.1 Mobile Application Configuration
 - 3.6.2 Admin Dashboard Configuration
 - 3.6.3 IoT Device (Collar) Configuration
 - 3.6.4 Security Configuration
 - 3.6.5 Performance Optimization
4. Hardware Infrastructure
 - 4.1 Central Control Unit
 - 4.1.1 Introduction
 - 4.1.2 Hardware Components
 - 4.1.3 Firmware

- 4.2 3D-Designs
- 4.3 PCB / Circuit Design
 - 4.3.1 Sensors
 - 4.3.1.1 GPS Module (NEO-8M)
 - 4.3.1.2 MPU6050 IMU (Activity & Respiratory Sensing)
 - 4.3.1.3 MLX90614 Temperature Sensor
- 4.4 Actuators
 - 4.4.1 KY-006 Piezo Buzzer
- 5. Activity, Impact & Geofence Alert System Using Sensor Data
 - 5.1 Introduction
 - 5.2 Detection Logic
- 6. Testing
 - 6.1 Software Testing
 - 6.2 Hardware Testing
- 7. Source Code Repositories
- 8. Appendix
 - 8.1 Bill of Materials & Budget
 - 8.2 Key Dependencies

1. Introduction

1.1 Project Overview

PetGuard Pro is a smart, IoT-based pet monitoring and safety system built around a wearable harness unit. It gives pet owners real-time visibility into their pet's location, movement, and basic vital signs, and raises alerts when the pet leaves a designated safe zone or when its vitals move outside a personalised healthy range. The system integrates real-time GPS location tracking, non-contact temperature sensing, motion/activity classification, and respiratory rate estimation, using an ESP32 microcontroller as the collar's central control unit. A mobile application gives the pet owner day-to-day monitoring and control, while a separate web-based admin dashboard supports staff who manage users and hardware.

1.2 Objective and Scope

Project objectives, as defined by the team:

- Real-time GPS location tracking of the pet
- Continuous health and activity monitoring using onboard sensors
- Geo-fence and emergency alert generation
- Remote buzzer / locate-pet activation
- A mobile application for everyday monitoring and control
- A cloud backend that can scale from prototype to production

The manual covers four engineering domains, corresponding to the four folders in the repository: hardware & embedded firmware, the mobile application, the admin web dashboard, and the Firebase cloud backend that connects the three. Each section below describes the component's responsibilities, architecture, key data structures and design rationale, followed by cross-cutting sections on data flow, security posture, testing, and hardware cost.

1.3 Solution Architecture

PetGuard Pro follows a modular, distributed IoT architecture that combines hardware, firmware, and a cloud platform to enable real-time pet monitoring and safety management. The wearable harness unit is the only data source; the Firebase cloud backend mediates all communication; and two client applications consume and manage that data.

1. Device Layer

User Devices:

- Pet Owner Mobile Application: live map, geofence editor, health dashboard, activity dashboard, alerts, and profile management. Also contains an in-app admin view.
- Admin Web Dashboard: a separate operations console for staff, gated by a Firestore "admins" collection, for user/harness management and support tickets.
- Communication with the backend is established via the native Firebase SDKs (Realtime Database, Firestore, Authentication, Cloud Messaging) using real-time listeners.

2. API & Backend Layer

API & Data Services:

- The ESP32 firmware does not use the Firebase SDK; it speaks directly to the Firebase Realtime Database REST API over HTTPS using the Arduino HTTPClient library, authenticating with a database secret.
- Responsible for handling sensor data from the collar, classifying activity/respiratory events on-device, and pushing telemetry and alerts to the cloud.

Database:

- Firebase Realtime Database stores dynamic, high-frequency data: live GPS coordinates, health readings, activity telemetry, and buzzer commands.
- Firebase Firestore stores static/structured data: user profiles, pet profiles, admin records, and support tickets.

3. Cloud & Messaging Layer

Firebase Cloud Platform:

- Acts as the core cloud backend for real-time database storage, structured document storage, authentication, and communication.
- Firebase Cloud Messaging delivers push notifications (e.g. geofence breach, impact detection, out-of-range vitals) to the mobile app.

4. Edge Device Layer

ESP32-WROOM-32 Microcontroller (Central Control Unit):

- Interfaces with all sensors and modules (GPS, MPU6050 IMU, MLX90614 temperature sensor, KY-006 buzzer).
- Executes onboard logic to classify activity (resting/walking/running/impact), estimate respiratory rate, and detect falls/impacts.
- Sends telemetry periodically to Firebase over WiFi/HTTPS and polls for remote buzzer commands.

5. Sensor & Module Layer

Sensors and Modules Connected to the ESP32:

- NEO-8M GPS Module – Provides latitude/longitude/HDOP for real-time pet location tracking.
- MPU6050 IMU (6-axis accelerometer + gyroscope) – Drives activity classification, step counting, impact/fall detection, and respiratory-rate estimation.
- MLX90614 Non-Contact Infrared Temperature Sensor – Monitors the pet's body temperature without needing to sit directly against skin.
- Piezo Buzzer – 1.2 kHz "dog-friendly" tone for remote locate/alert activation, with a 120-second safety cutoff.
- 1500 mAh 3.7 V Li-Po Battery + USB-C charging + buck converter – Portable power with a battery-voltage monitor on an ADC pin.

6. Data Flow & Communication Protocols

Sensor Data Acquisition:

- The ESP32 polls IMU/GPS/temperature sensors every 20 ms and runs classification logic continuously.
- Current-state values (location, health, activity) are PATCHed to Firebase on their own timers (5–6 second intervals); history records are POSTed on separate timers.
- The firmware does not use the Firebase SDK. It calls the Realtime Database REST API directly with GET/PATCH/POST verbs, authenticated by a database secret.

User Interaction:

- The mobile app uses real-time Firebase listeners (not polling) to reflect location, health, and activity updates instantly.
- The mobile app writes buzzer commands to `pets/{petId}/commands/buzzer`, which the firmware polls and executes.

The Admin Dashboard provides:

- User and harness/hardware management
- Aggregate system/usage metrics and charts
- A support-ticket desk for responding to user issues

7. Cloud Integration

Firebase Hosting: the Admin Dashboard (React/Vite build) is deployed via Firebase Hosting, with Firebase configuration loaded from Vite environment variables.

Mobile App Deployment: the Flutter app targets Android, iOS, web, and desktop; distribution channel.

1.4 Data Flow

1. Sensors on the collar collect real-time data: GPS coordinates, accelerometer/gyroscope readings, and body temperature.
2. The ESP32 processes this data on-device: classifying activity (resting/walking/running/impact), counting steps, and estimating respiratory rate from Z-axis accelerometer oscillations.
3. Current-state data (location, health, activity) is PATCHed to the Firebase Realtime Database on a fixed schedule; history records are POSTed separately for later charting.

- 4. The mobile app receives geofence breach alerts, impact alerts, and out-of-range vital alerts in real time via Firebase listeners and local notifications.
- 5. The admin dashboard displays aggregate system metrics, registered users/harnesses, and support tickets, sourced from Firestore and the Realtime Database.
- 6. All data is stored in the cloud using Firebase Realtime Database for dynamic telemetry and Firestore for structured records, allowing remote monitoring and longer-term analysis. During WiFi outages, readings are buffered to the ESP32's flash (NVS) and replayed once connectivity returns.

1.5 Security

- Pet owners authenticate via Firebase Authentication (email/password) and are scoped to the pet referenced by their users/{uid}.selectedPetId document. They cannot see other households' data by construction of the app's queries.
- Admin users are distinguished purely by the existence of a matching document in the admins Firestore collection; the dashboard signs a user back out immediately if no such document exists after login.
- The collar itself authenticates to the Realtime Database using a static database secret embedded in firmware (see note below).
- The Admin Dashboard and Mobile App load Firebase configuration from environment variables / generated platform files rather than hard-coding secrets in application source.
- The admin dashboard performs a genuine server-side authorization check (Firestore document lookup) rather than trusting a client-side role flag, and forcibly signs out non-admins.
- The buzzer's 120-second safety watchdog limits the blast radius of a stuck or malicious "on" command.

2. Software Infrastructure

2.1 Frontend

2.1.1 Key Features

- Real-Time Pet Location Tracking: live tracking of the pet using GPS data, shown on an interactive map with an accuracy radius.
- Geofencing: circular and polygon safe-zones, with optional day/time-of-day schedules.
- Health Monitoring: personalised temperature and respiratory-rate thresholds derived from the pet's profile.
- Activity Monitoring: resting/walking/running/impact classification, step counting, and an activity history feed.
- Alerts & Notifications: local + push notifications for geofence breaches, impact detection, and out-of-range vitals.
- Remote Locate/Alert: buzzer activation from the mobile app, with a firmware-side safety cutoff.
- Historical Data Visualisation: location history, health history, and activity history, viewable over time.

2.1.2 Technology Stack

Component	Technology / Package	Purpose / Role
Language	Dart (mobile), TypeScript (admin)	Main languages for the Flutter app and React admin dashboard
Mobile Framework	Flutter (package smart_pet_app)	Cross-platform pet-owner app — Android, iOS, web, Windows, Linux, macOS
Mobile State Mgmt	flutter_riverpod	Theme state and cross-cutting providers
Mobile Local Storage	hive / hive_flutter, shared_preferences	Offline persistence for geofences, location history, and settings
Web Framework	React 19 + TypeScript, built with Vite 8	Admin operations console
Web Styling	Tailwind CSS 3	Utility-first styling, teal (#00897B) primary colour matching the mobile app
Charts	fl_chart (mobile), Recharts (admin)	Health/activity history charts and dashboard metrics

Component	Technology / Package	Purpose / Role
Maps	google_maps_flutter, geolocator	Live map, geofence editing, and location history rendering
Notifications	flutter_local_notifications, Firebase Cloud Messaging	In-app and push alerting

2.1.3 UI Design

Both clients share a teal-based Material 3 (mobile) / Tailwind (admin) design language, giving a consistent brand identity across the pet-owner app and the operations console.

Token	Light Theme	Dark Theme
Primary / seed colour	#009688 (teal)	#4DD0E1 (light teal / cyan)
Scaffold background	#F7FAFA	#0F1515
App bar	White, black text, no elevation	#121A1A, white text, no elevation
Card	White, 16 px corner radius, no elevation	#172020, 16 px corner radius, no elevation
Buttons	Teal fill, white text, 12 px radius	Light-teal fill, black text, 12 px radius
Inputs	White fill, 12 px radius, teal focus border	#1E2929 fill, 12 px radius, light-teal focus border

Design principles: consistent 12–16 px corner radii across cards, buttons and inputs; zero elevation, with colour/contrast used for hierarchy instead of shadow; a single accent hue (teal) carried through both light and dark palettes and across both mobile and admin surfaces.

Dashboard



Buddy

Pet ID: default_pet

Live Telemetry Vitals



61 br/min

Respiratory Rate
Elevated breathing



29.1 °C

Body Temp
Normal range



405 steps

Steps Walked
8% of daily goal



RESTING

Active State
Sleeping / Inactive

GPS Safe Zone Monitor



GPS Signal Connected

Coordinates: 7.2548, 80.5917

Track Live Location

Behavior Analytics



Behavior Summary PDF

Compile and export all vital logs, daily classification ratios, and GPS movement footprints for Buddy.

Generate Today's Report

Dog Behaviour Classification

Daily Step Progress

405 / 5000 steps

Activity Ratios Distribution

Resting: 40%

Walking: 40%

Running: 20%

Health Telemetry Summary



24 breaths/min

Average Resp. Rate

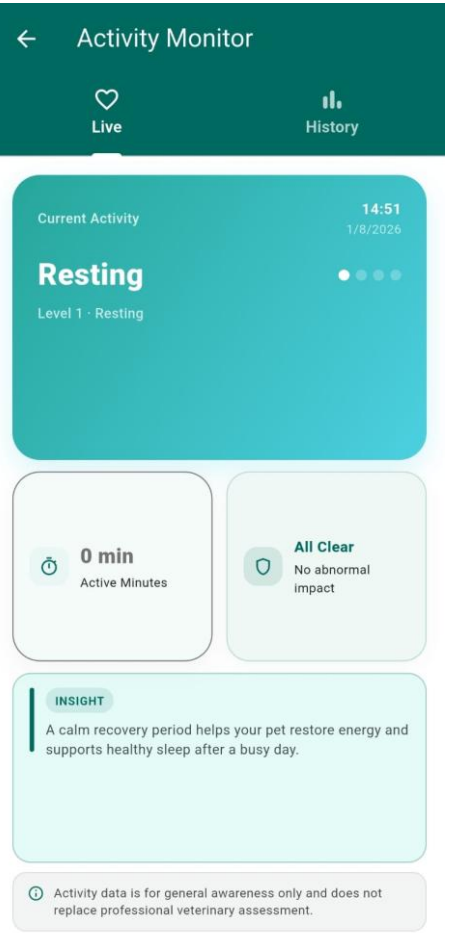
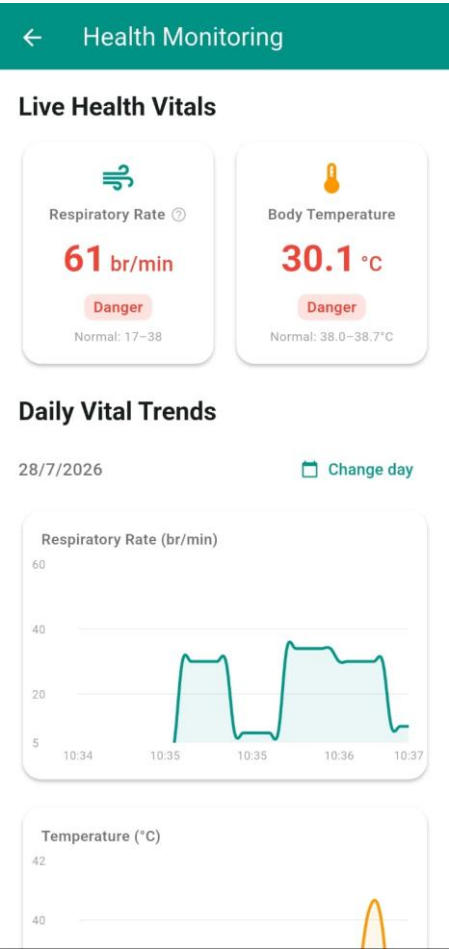
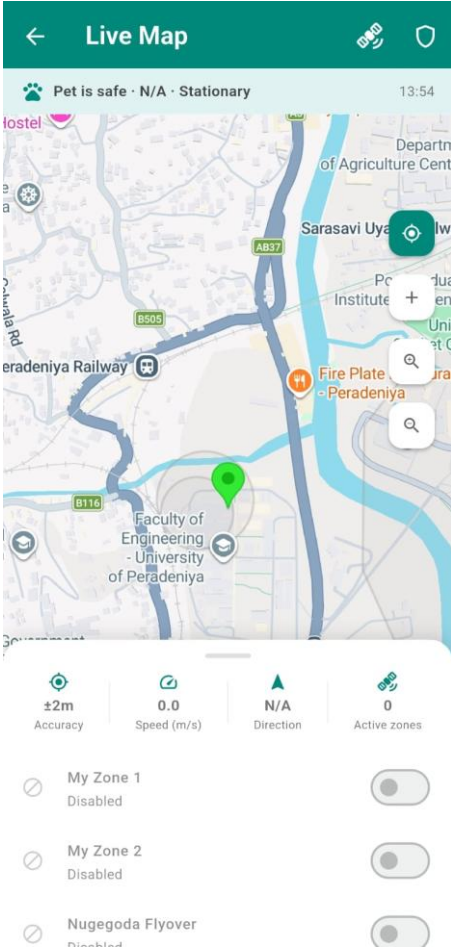
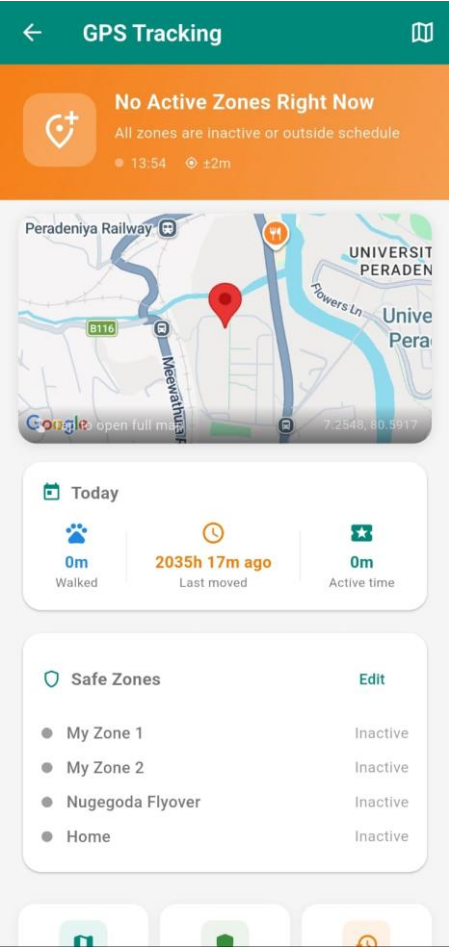


38.6 °C

Average Body Temp

HomeAnalyticsFeatures

HomeAnalyticsFeatures



2.2 Backend

2.2.1 Introduction

The backend is Firebase, connecting the ESP32 collar, the Flutter mobile app, and the React admin dashboard. It handles telemetry ingestion, structured data storage, authentication, and push notification delivery, with a clear separation of concerns across three Firebase services.

2.2.2 Key Features

- Real-Time Telemetry Ingestion: accepts location, temperature, respiratory-rate, and activity data pushed by the collar over HTTPS.
- Structured User/Pet Records: Firestore documents for owners, pets, admins, and support tickets.
- Authentication & Role-Based Access: Firebase Authentication for identity; admin rights gated by document presence in a Firestore admins collection.
- Push Notification Delivery: Firebase Cloud Messaging routes alerts (geofence, impact, vitals) to the owner's device.
- Hybrid Data Storage Strategy: Realtime Database for dynamic/high-frequency telemetry; Firestore for static/structured documents.

2.2.3 Technology Stack

Component	Technology	Purpose / Role
Realtime Telemetry Store	Firebase Realtime Database	Stores and streams live GPS, health, and activity data; buzzer commands
Structured Data Store	Firebase Firestore	Stores user, pet, admin, and support-ticket documents
Authentication	Firebase Authentication	Email/password sign-in for owners and admins
Push Notifications	Firebase Cloud Messaging	Delivers alerts to the mobile app
Hosting	Firebase Hosting	Serves the built React admin dashboard
Firmware ↔ Cloud Transport	Arduino HTTPClient over HTTPS	Direct REST calls from the ESP32 to the Realtime Database (no SDK)

2.2.4 Modular Architecture and Data Flow

Realtime Database layout (as written by the firmware):

```
pets/ {petId}/  current_location { latitude, longitude, accuracy, heading, timestamp }  location_history/ { ...same fields, one entry per
push }  health { temperature, respiratoryRate, timestamp }  health_history/ { ...same fields, one entry per push }  activity/  current
{ activity_type, magnitude, impact_detected, impact_severity, step_count, active_minutes, accelerometer{x,y,z}, gyroscope{x,y,z},
timestamp }  history/ { ...same shape, one entry per push }  commands/  buzzer { active: bool } <- set by the mobile app, read by
firmware
```

Firestore collections (as used by the apps):

Collection	Purpose	Consumers
users/{uid}	Owner profile; selectedPetId links the signed-in user to a collar/pet	Mobile app (auth/authorization module)
admins/{uid}	Presence of a document grants admin rights	Admin dashboard role gate, mobile app admin view
pets / dog profile documents	Pet metadata (breed, size, age, coat, activity level) used to personalise health thresholds	Mobile app health module
support tickets	Support ticket records	Admin dashboard support desk

Because the firmware writes ISO-like timestamp strings while the Realtime Database also injects server-side timestamps, the mobile app's models explicitly handle both an integer millisecond epoch and a string timestamp, including a defensive fix-up for a known firmware quirk where dashes appear in the time portion of the ISO string (e.g. 2026-03-04T13-36-17).

2.2.5 Firebase REST Interface (Firmware & Cloud)

The ESP32 firmware does not use a Firebase SDK; it calls the Realtime Database REST API directly. Three verbs are used consistently across the firmware:

HTTP Verb	Firmware Function	Purpose
GET	firebaseGet()	Poll pets/{petId}/commands/buzzer for a remote buzzer on/off command
PATCH	firebasePatch()	Overwrite a "current state" node (current_location, health, activity/current) — always the latest value only
POST	firebasePost()	Append a new timestamped record to a history list (location_history, health_history, activity/history)

Authentication: the collar authenticates with a database secret passed as a query parameter (see §1.5/§3.3.2 for the security caveats of this approach).

Client SDK usage: the mobile app, by contrast, uses the native Firebase SDKs (firebase_database, cloud_firestore, firebase_auth) with real-time listeners rather than polling.

3. Cloud Deployment Architecture and Procedure

3.1 Architecture Overview

3.1.1 Multi-Tier Architecture Design

PetGuard Pro follows a three-tier architecture deployed on Firebase:

- Presentation Tier: Flutter mobile app (owner) and React admin dashboard (staff)
- Application / Integration Tier: Firebase Realtime Database REST API (device-facing) + native Firebase SDKs (client-facing)
- Data Tier: Firebase Realtime Database for live telemetry, Firestore for structured records

3.1.2 Cloud Infrastructure Components

- Firebase Realtime Database: real-time IoT telemetry streaming and buzzer command relay
- Firebase Firestore: user, pet, admin, and support-ticket documents
- Firebase Authentication: identity for pet owners and admin staff
- Firebase Cloud Messaging: push notification delivery
- Firebase Hosting: serves the built admin dashboard
- Firebase Storage: referenced for any user/pet media assets

3.2 Firebase Project Setup

3.2.1 Firebase Account & Project Configuration

Project ID: petguardpro-efda9

Pricing Plan: Spark (free tier)

3.2.2 Realtime Database & Firestore Creation

Realtime Database URL: petguardpro-efda9-default-rtdb.asia-southeast1.firebaseio.com

Realtime Database Region: asia-southeast1 (Singapore)

3.2.3 Hosting Setup for Admin Dashboard

Hosting Site Name/URL: petguardpro-efda9.web.app (default domain), also accessible via petguardpro-efda9.firebaseio.com

Deploy Command: firebase deploy --only hosting

3.3 Environment & Security Configuration

3.3.1 Environment Variables Setup

The Admin Dashboard loads Firebase configuration entirely from Vite environment variables (import.meta.env.VITE_FIREBASE_*), keeping credentials out of source. The Mobile App loads Firebase configuration from generated platform files (google-services.json / GoogleService-Info.plist).

3.3.2 Security Rules Configuration

Realtime Database Security Rules

```
{
  "rules": {
    ".read": true,
    ".write": true,
    "pets": {
      "$petId": {
        "health_history": {
          ".indexOn": ["timestamp"]
        }
      }
    }
  }
}
```

Rules are currently open for development/testing; production deployment would require scoping .write to authenticated device requests (e.g. via Firebase Auth custom tokens per collar) to prevent unauthorized writes.

Cloud Firestore Security Rules

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.time < timestamp.date(2026, 10, 21);
    }
  }
}
```

The above ruleset reflects Firestore's default test-mode configuration, generated automatically at database creation. It permits unrestricted read and write access to all documents, which was sufficient for development and integration testing during this phase of the project. Access is not yet scoped to authenticated users or specific document ownership (e.g., restricting a pet owner's access to only their own pet's records).

3.3.3 Authentication Configuration

- Sign-in method: Firebase Authentication, email/password
- Pet owners are scoped to the pet referenced by their users/{uid}.selectedPetId document
- Admin rights are granted purely by the existence of a matching document in the admins Firestore collection

3.4 Database, Frontend and Backend Deployment

3.4.1 Realtime Database & Firestore Setup

See §2.2.4 for the Realtime Database node layout and Firestore collection structure written/read by the firmware and apps.

3.4.2 Authentication & Cloud Messaging Configuration

Notifications (e.g., collar misalignment alerts) are handled entirely through local notifications on-device via the flutter_local_notifications package, triggered when the app processes new sensor data while running or backgrounded. As a result, no FCM sender ID, server key, or push-notification channel configuration was required for the current implementation.

Push notifications when the app is fully closed are identified as a limitation of the current system and a candidate for future work, requiring FCM integration alongside a backend trigger (e.g., a Cloud Function watching the Realtime Database for out-of-range vitals) to notify users regardless of app state.

3.4.3 Admin Dashboard Deployment Process

Build: `npm run build`

Deploy: `firebase deploy --only hosting`

3.4.4 Mobile App Deployment Process

The application is versioned at 1.0.0+1 (pubspec.yaml) and uses the placeholder bundle identifier com.example.smartPetApp. The flutter_launcher_icons package is configured with a custom app icon (assets/icon/icon.png).

For demonstrations, the app was deployed via `flutter run` debug builds on connected devices, or a `flutter build apk --release` sideloaded APK.

3.5 Service Management

3.5.1 Application Monitoring

Firebase Console monitoring is used for the Realtime Database and Firestore usage tabs (Console → Usage and billing), tracking read/write/download operations against the Spark plan's free-tier quotas.

3.5.2 Logging Configuration

Firmware Logging (ESP32): Serial logging runs at 115200 baud (Serial.begin(115200)), initialized at boot. Log output follows several conventions:

- A banner block is printed at boot and on each live-data cycle, framed by '=' separator lines, identifying the system and reporting boot-time status.
- Each live data cycle prints a structured, numbered section report (e.g. "[1] GPS Data", "[2] Temperature Data (MLX90614)", "[3] Heart Rate + SpO2 + Activity", "[4] Data Sent to Firebase"), with each section underlined by a dashed separator and individual readings aligned as " Label : value" pairs for readability over serial monitor.
- Flash buffering and sync events are logged distinctly, allowing offline/recovery behaviour (see §3.5.3) to be traced directly from serial output.
- Sensor and connectivity failures are logged with explicit messages , and implausible sensor readings are flagged rather than silently discarded.

Mobile App Logging:

Client-side logging uses Flutter's debugPrint() throughout, with emoji-prefixed markers for quick visual scanning in the debug console.

3.5.3 Offline Resilience (Flash Buffering)

Every push function (pushGPS, pushHealth) falls back to saveToFlash() when the corresponding firebasePatch()/firebasePost() call fails. Buffered JSON entries are stored under two keys in the ESP32's non-volatile Preferences namespace (gps_queue, health_queue), capped at 8,000 bytes, and are replayed to Firebase automatically once connectivity returns (syncAllFlashBuffers()). This means short outages do not create gaps in the pet's location or health history.

3.6 Client Configuration

3.6.1 Mobile Application Configuration

Firebase configuration is managed via the FlutterFire CLI-generated lib/firebase_options.dart file, defining platform-specific FirebaseOptions (Android, iOS, Web, macOS, Windows) selected at runtime through DefaultFirebaseOptions.currentPlatform. This supersedes the older pattern of manually placing google-services.json (Android) / GoogleService-Info.plist (iOS) in platform folders. The app has a single build pointing at the petguardpro-efda9 Firebase project.

3.6.2 Admin Dashboard Configuration

The Admin Dashboard's Firebase configuration is loaded from a .env file (excluded from version control) and consumed in src/firebase.ts via Vite's import.meta.env mechanism. The following environment variables are used:

VITE_FIREBASE_API_KEY

VITE_FIREBASE_AUTH_DOMAIN

VITE_FIREBASE_PROJECT_ID

VITE_FIREBASE_STORAGE_BUCKET

VITE_FIREBASE_MESSAGING_SENDER_ID

VITE_FIREBASE_APP_ID

VITE_FIREBASE_DATABASE_URL

The dashboard initializes the Firebase Auth, Firestore, and Realtime Database SDKs (firebase/auth, firebase/firestore, firebase/database) from this shared configuration.

3.6.3 IoT Device (Collar) Configuration

Configured in esp32-firmware/src/config.h: WiFi SSID/password, Firebase database URL and secret, pin assignments, and detection thresholds (see §4.1.2 and §4.1.3).

3.6.4 Security Configuration

- API Authentication: Firebase Authentication tokens (client), database secret (device — flagged for rotation, see §1.5)
- HTTPS Enforcement: all client and device communications use HTTPS
- Access Control: Firestore document-based scoping (owner → pet, admin → admins collection)

3.6.5 Performance Optimization

- Real-time listeners (not polling) on the client side minimise unnecessary reads
- Firmware push intervals are tuned per data type (20 ms sensor read; 5–7 s Firebase pushes) to balance responsiveness against bandwidth/battery use
- Local Hive caching of geofences and location history reduces redundant reads and supports offline use

4. Hardware Infrastructure

4.1 Central Control Unit

4.1.1 Introduction

The Central Control Unit (CCU) of PetGuard Pro's collar is built around the ESP32-WROOM-32 microcontroller, which serves as the core processing and communication hub. It is responsible for reading data from onboard sensors — the GPS module, MLX90614 temperature sensor, and MPU6050 IMU — classifying that data into activity, impact, and respiratory-rate events, and driving the KY-006 buzzer on command. The ESP32 was chosen for its built-in WiFi, sufficient processing power for on-device sensor fusion, low power consumption, and low cost, appropriate for a wearable prototype.

4.1.2 Hardware Components

- ESP32-WROOM-32 (esp32doit-devkit-v1)

Acts as the Central Control Unit (CCU) of the collar. Offers built-in WiFi, dual-core processing, multiple GPIOs, and ADC support, making it suitable for real-time sensor fusion and HTTPS communication with the cloud.

- MPU6050 IMU

A 6-axis accelerometer + gyroscope used for activity classification, step counting, impact/fall detection, and — via Z-axis oscillation analysis — respiratory rate estimation. Communicates via I²C.

- NEO-8M GPS Module

Provides latitude/longitude/HDOP for real-time pet location tracking. Communicates over UART (Hardware Serial 2, 9600 baud), decoded with the TinyGPS++ library.

- MLX90614 Non-Contact Infrared Temperature Sensor

A non-contact temperature sensor, chosen over contact thermistors so it does not need to sit directly against skin — more comfortable and reliable on a moving animal. Shares the I²C bus with the MPU6050.

- KY-006 Piezo Buzzer

Used as a remote locate/alert actuator. Fixed at a 1.2 kHz "dog-friendly" tone, driven via an LEDC PWM channel, with a hard-coded 120-second safety cutoff (BUZZER_SAFETY_MS) to prevent it being left on indefinitely.

- 2500 mAh 3.7V Li-Po Battery + USB-C Charging + Buck-Boost Converter

Portable power source for the collar, with a battery-voltage monitor on an ADC pin for remaining-charge estimation.

						
ESP32 Board	NEO8M GPS	MPU6050 IMU	MLX90614	KY-006 Piezo Buzzer	TP4056	TPS63020

Pin & interface map (from config.h):

Signal	Pin / Interface	Notes
GPS RX / TX	GPIO 16 / 17	Hardware Serial 2, 9600 baud
I ² C bus (IMU + Temp)	GPIO 21 (SDA) / 22 (SCL)	100 kHz clock; shared by MPU6050 and MLX90614
Buzzer control	GPIO 25	Direct drive of the KY-006 "S" pin via LEDC PWM channel 0
Battery monitor	GPIO 34 (ADC)	Analog read for remaining charge estimation

4.1.3 Firmware

The firmware (main.cpp, ~790 lines, version tag "v6.4 — WiFi + Respiratory Rate") is a single-file Arduino sketch built with PlatformIO, structured around cooperative, non-blocking timers driven from loop() rather than an RTOS task model.

Startup sequence (setup()):

- Initialise the buzzer PWM channel and serial monitor
- Open the flash-based offline queue (Preferences, namespace "petguard")
- Bring up the I²C bus and initialise the MPU6050 and MLX90614, logging whether each responds
- Start GPS UART and connect to WiFi
- If WiFi is available at boot, immediately flush any previously queued offline readings

Main loop timers:

Timer	Interval	Action
Sensor read	20 ms	Poll IMU/GPS/temperature sensors continuously
Current state push	5,000 ms	PATCH latest activity snapshot to Firebase
History save	7,000 ms	POST an activity history record
Health push	5,000 ms	PATCH + POST temperature and respiratory rate
GPS push	6,000 ms	PATCH + POST current and historical location
Buzzer command check	3,000 ms	GET the buzzer command node from Firebase
Connection check	30,000 ms	Verify WiFi is still connected; reconnect if not
Flash sync	30,000 ms	Attempt to flush any queued offline records

Sensor Processing Logic

Activity Classification:

classifyActivity() combines accelerometer magnitude and gyroscope magnitude against three configurable threshold pairs (impact / running / walking, all in config.h) to bucket the pet's current state into impact, running, walking, or resting. An impact is also flagged independently if acceleration alone exceeds the impact threshold by a wide margin, so a hard fall is still caught even without a matching gyroscope spike.

Step Counting:

updateStepCount() implements a peak-detection step counter: a step is registered when acceleration magnitude crosses above 11.5 m/s² on a rising edge relative to the previous sample.

Respiratory Rate Estimation:

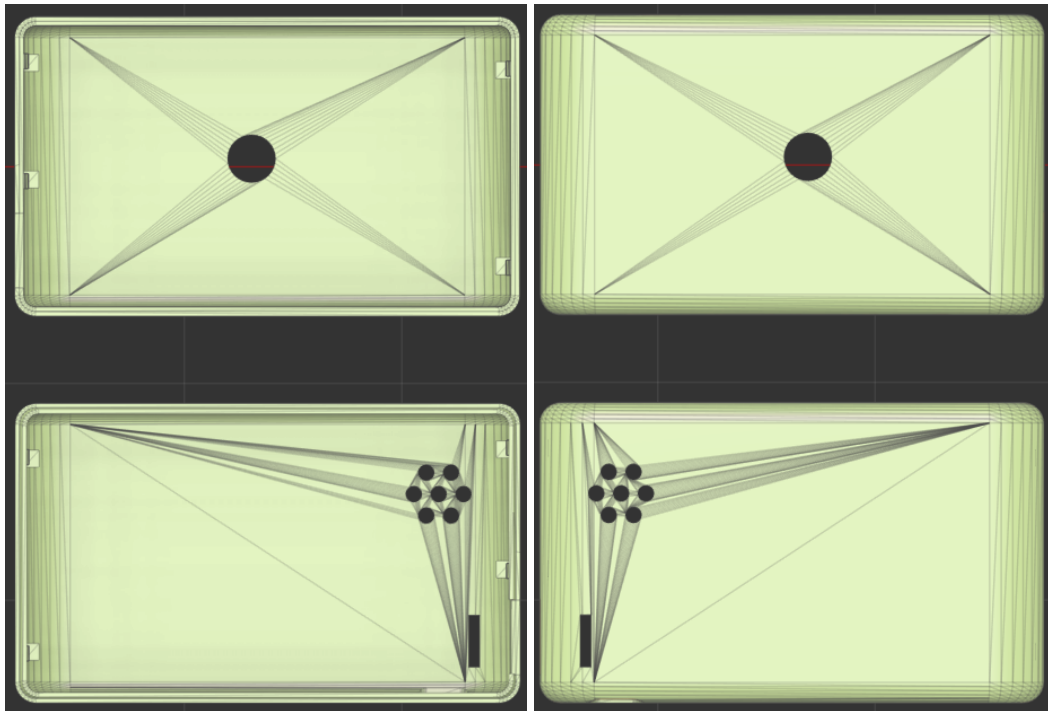
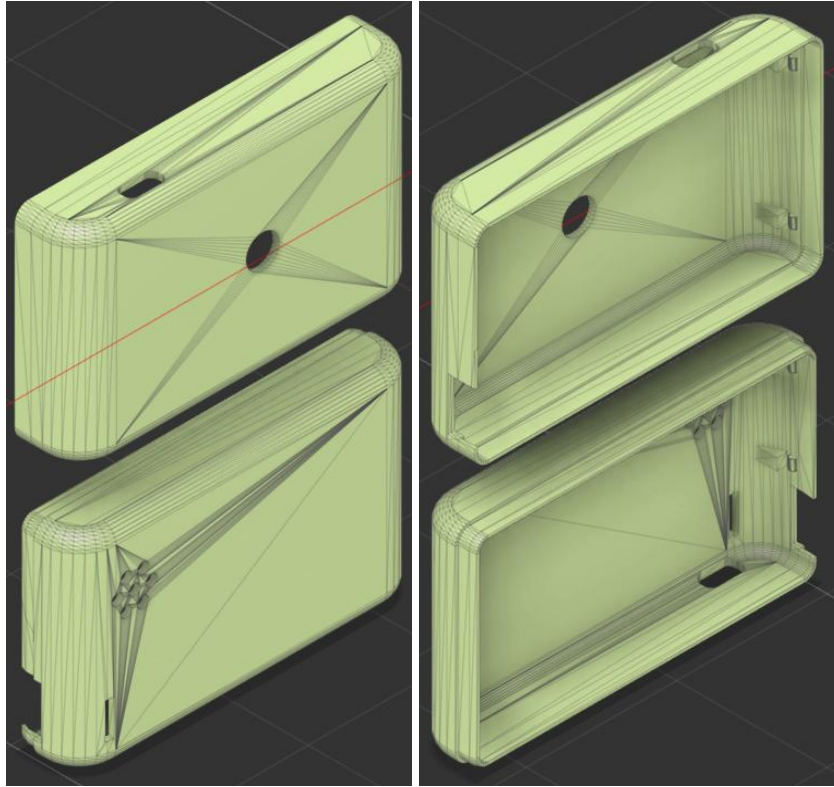
The accelerometer's Z-axis is used to detect small oscillations caused by chest expansion. An exponential moving average (alpha = 0.1) smooths high-frequency motion noise; the code counts rising zero-crossings through the smoothed baseline that exceed a minimum deviation (0.04 m/s²) within a rolling 30-second window, and doubles the count to produce breaths-per-minute.

GPS Accuracy Estimation:

getAccuracy() derives an approximate horizontal accuracy in metres from the GPS module's HDOP value (accuracy ≈ HDOP × 2), clamped to a 2–50 m range, so the mobile app can show a realistic accuracy radius rather than a raw HDOP number.

4.2 3D-Designs

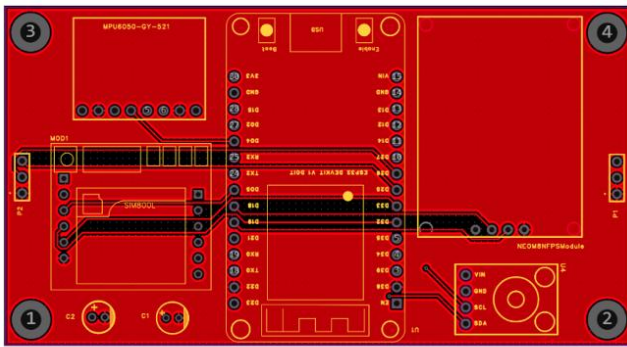
Covers the enclosure designed to mount the ESP32, sensors, buzzer, and battery securely on the pet.



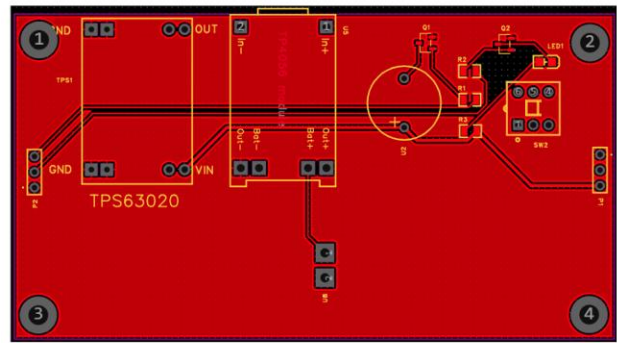
4.3 PCB / Circuit Design

PCB layout

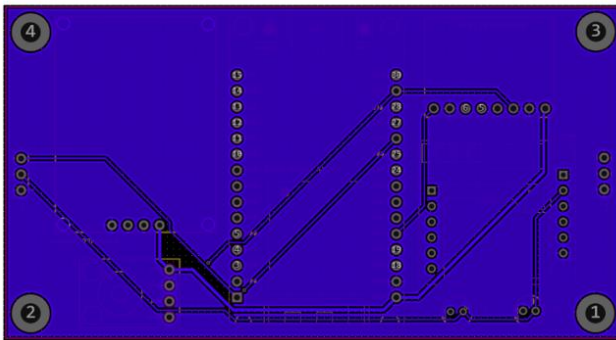
PCB Layout - Top Side



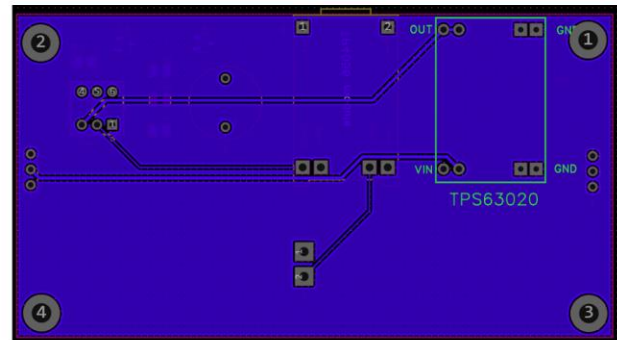
PCB Layout - Top Side



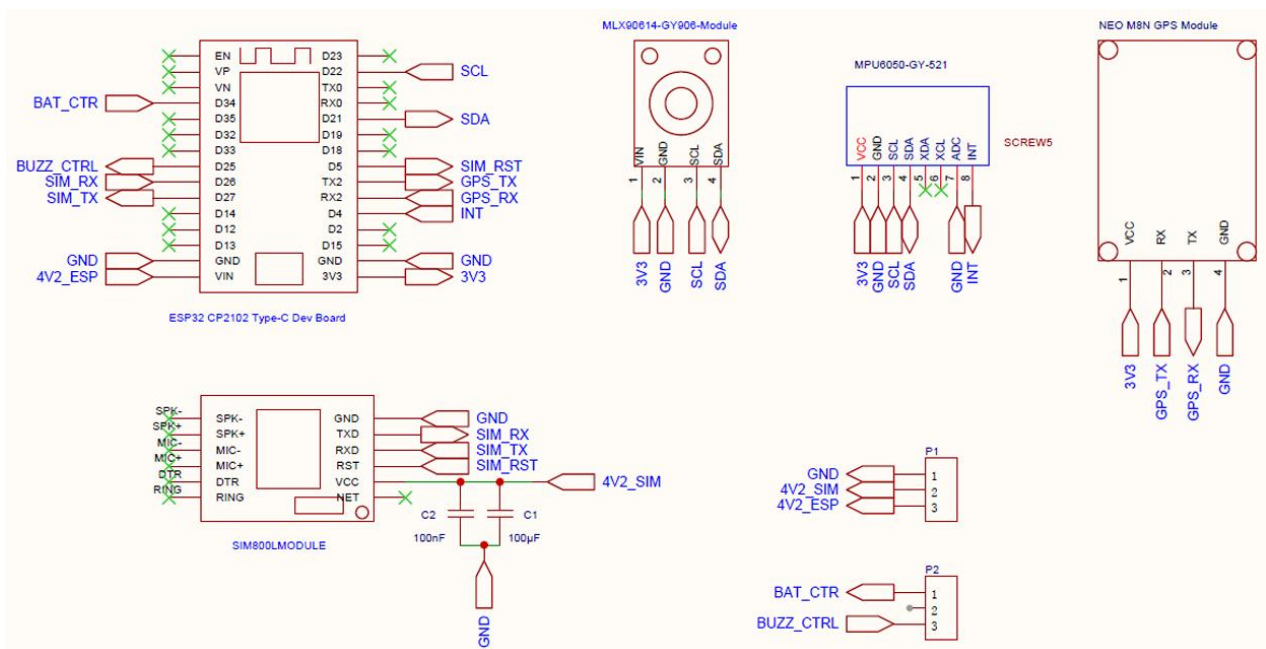
PCB Layout - Bottom Side

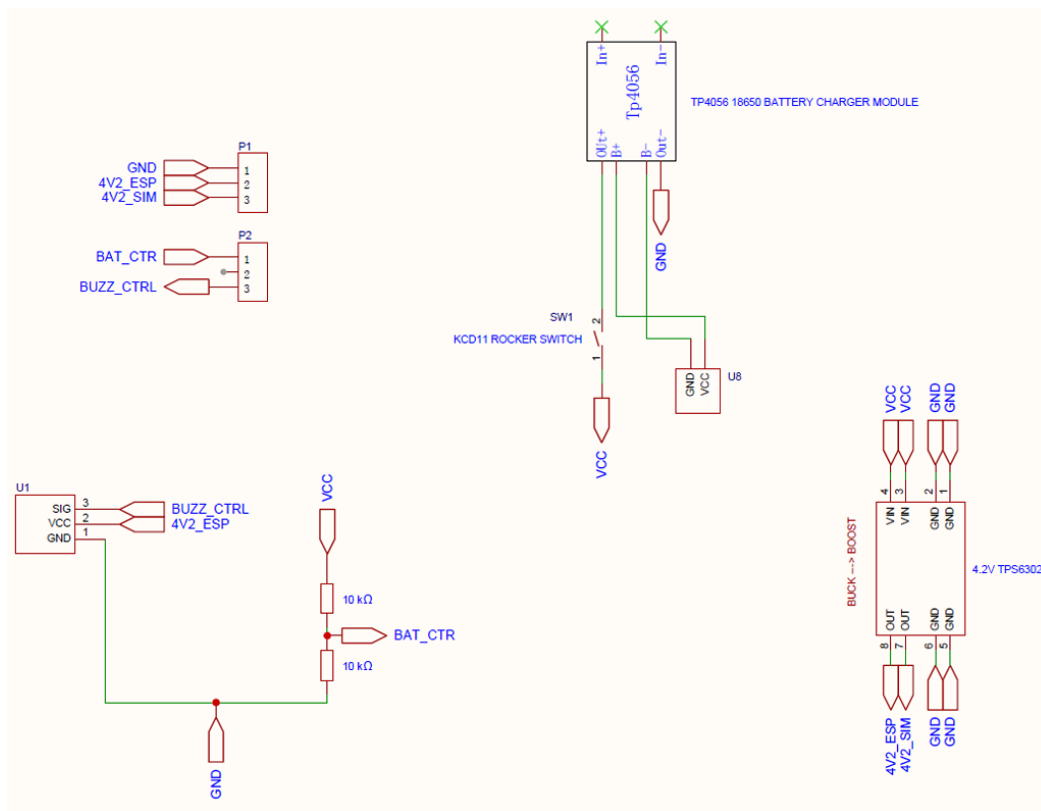


PCB Layout - Bottom Side



Schematic Diagram





4.3.1 Sensors

4.3.1.1 GPS Module (NEO-8M)

- The GPS tracking system uses a NEO-8M GPS module that continuously receives satellite data and transmits location information (latitude, longitude, HDOP) to the ESP32 via UART.
- The ESP32 reads the GPS data using the TinyGPS++ library, derives an approximate accuracy from HDOP, and packages it into JSON. This location data is periodically pushed to Firebase using the HTTPS REST API.
- The real-time location is displayed on the mobile app's live map, with a geofence overlay and an accuracy radius.

4.3.1.2 MPU6050 IMU (Activity & Respiratory Sensing)

- The MPU6050 is connected to the ESP32 via the I²C interface and is used to monitor the pet's motion and orientation. It detects acceleration/gyroscope patterns consistent with resting, walking, running, or an impact.
- The firmware filters the signal and compares it against configurable thresholds; an impact that exceeds the threshold by a wide margin is flagged even without a matching gyroscope spike.
- The same Z-axis data feeds the respiratory-rate estimator (see §4.1.3). Activity and health data are pushed to Firebase and can trigger local notifications on the mobile app (e.g. on impact detection).

4.3.1.3 MLX90614 Temperature Sensor

- The MLX90614 is connected to the ESP32 via the I²C bus (shared with the MPU6050) and periodically measures the pet's body temperature without contact.
- The ESP32 applies a personalised calibration (fur-type offset, and a dog-profile-based threshold from VitalThresholds.fromProfile()) before uploading the reading to Firebase via the HTTPS REST API.
- This data is displayed on the owner's health dashboard, with a personalised normal/caution/danger band based on the pet's size, age, facial structure, and activity level, and can help flag overheating or fever early.

4.4 Actuators

4.4.1 KY-006 Piezo Buzzer (Locate / Alert)

- The KY-006 piezo buzzer is used for on-collar locate/alert functionality. It is driven directly from the ESP32 via an LEDC PWM channel.
- When the owner sends a buzzer-on command from the mobile app, the ESP32 polls the commands/buzzer node (every 3 seconds) and activates the buzzer at a fixed 1.2 kHz "dog-friendly" tone.
- buzzerSafetyWatchdog() force-disables the buzzer after a fixed 120-second ceiling regardless of the stored command state, protecting the pet from a stuck-on or forgotten alarm.

5. Activity, Impact & Geofence Alert System Using Sensor Data

5.1 Introduction

The system uses an MPU6050 IMU integrated with an ESP32 microcontroller to monitor the pet's activity level and detect sudden impacts (falls), combined with GPS-based geofence monitoring for safe-zone breaches. Sensor data is processed locally on the ESP32 and on the mobile app (which mirrors the firmware's classification client-side), and alerts are surfaced through Firebase in real time, local device notifications, and direct on-collar audible feedback.

5.2 Detection Logic

Activity Classification

- Continuously compares accelerometer magnitude and gyroscope magnitude against three configurable threshold pairs (impact / running / walking) to classify the pet's current state as impact, running, walking, or resting.
- An impact is also flagged independently whenever acceleration alone exceeds the impact threshold by a wide margin, so a hard fall is still caught even without a matching gyroscope spike.
- Activity events are pushed to Firebase (current + history) and mirrored by the mobile app's activity module, which drives local notifications on impact detection.

Respiratory Rate Estimation

- Uses the accelerometer's Z-axis to detect small oscillations from chest expansion, smoothed with an exponential moving average ($\alpha = 0.1$).
- Counts rising zero-crossings through the smoothed baseline that exceed a 0.04 m/s^2 minimum deviation within a rolling 30-second window, doubled to give breaths-per-minute.
- Compared against a personalised threshold band (see below) to flag values outside the pet's expected normal range.

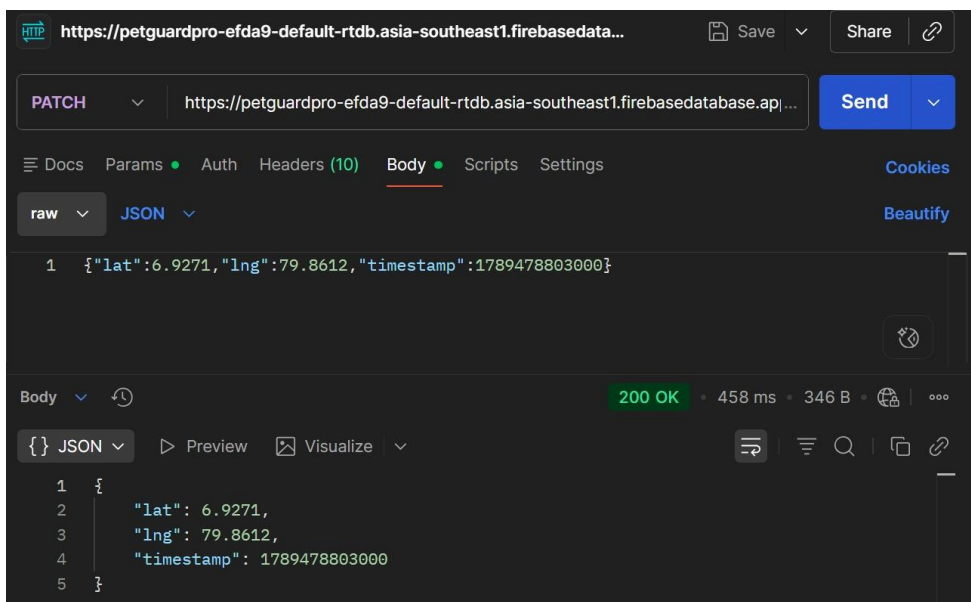
Geofence Breach Detection

- Geofence zones (circular or polygon) are defined by the owner in the mobile app and evaluated against live GPS coordinates.
- Polygon containment uses a standard ray-casting, point-in-polygon algorithm operating on longitude/latitude pairs.
- Each zone can carry an optional schedule (active days of week + time-of-day window), including correct handling of overnight windows that cross midnight (e.g. 22:00–06:00).
- A breach triggers a local/push notification to the owner.

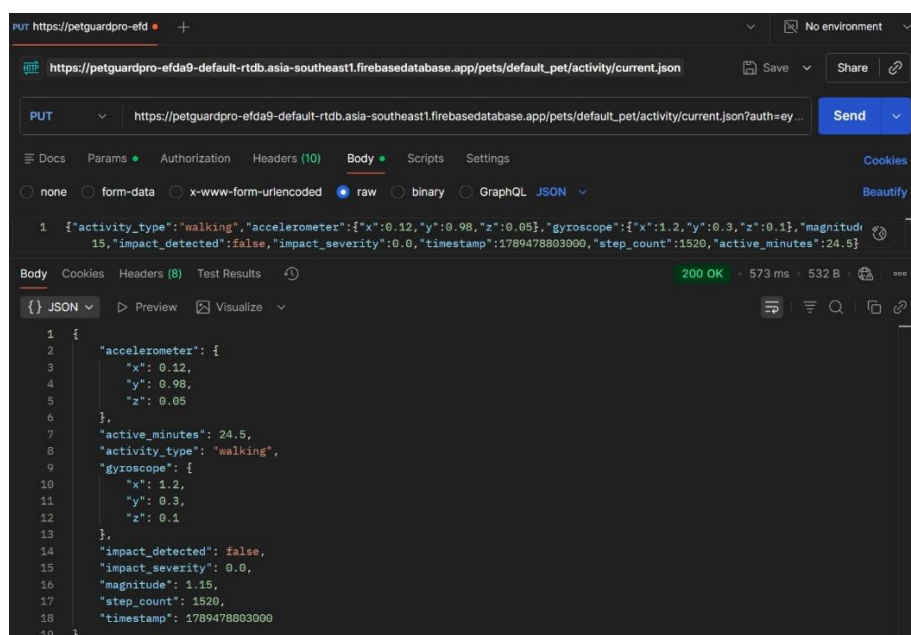
Personalised Vital Thresholds & Baseline Calibration

- Rather than a single fixed "normal range", `VitalThresholds.fromProfile()` derives a personalised normal/caution/danger band for temperature and respiratory rate from the pet's profile: size (small/medium/large/giant), age group (puppy/adult/senior), facial structure (flat-faced breeds run warmer and breathe faster), and activity level.
- This produces materially different thresholds for, say, a senior giant-breed dog versus an adult small dog, rather than a one-size-fits-all rule.
- Design intent follows an awareness-indicator philosophy — surfacing "outside normal range" rather than a clinical "danger" diagnosis — consistent with the system being a monitoring aid rather than a veterinary device.

Profile Factor	Effect on Temperature Band	Effect on Respiratory Band
Puppy	Wider tolerance, slightly warmer baseline	Higher normal range (+2 to +5 br/min)
Senior	Lower upper bound (reduced thermoregulation)	Higher upper bound
Flat-faced breed	Lower upper bound (overheats more easily)	Higher range across the board
Giant / Small size	Slightly cooler / warmer baseline respectively	Lower / higher range respectively



location



activity

6.1.2 Functionality Testing

- Ensured all connected modules (GPS, MPU6050, MLX90614) transmit data accurately to Firebase.
- Verified real-time activity, impact, and geofence-breach alerts via local notifications, push notifications, and the mobile app UI.
- Validated live GPS tracking updates and correct geofence containment logic, including overnight schedule windows.

6.1.3 User Interface Testing (Mobile App — Owner Side)

- Tested login, registration, and Firebase Authentication session handling.
- Verified correct rendering of real-time GPS coordinates and geofences on the map.
- Checked health/activity data visualisation (temperature, respiratory rate, activity charts, threshold bands).
- Validated alert/notification flow and acknowledgment.
- Tested responsive design across mobile, web, and desktop build targets.

6.2 Hardware Testing

6.2.1 GPS Module Testing

- Location Accuracy Test: compared GPS coordinates from the module with known reference locations.
- Signal Stability: evaluated GPS fix time and signal retention under different environments (open sky, urban, indoor).
- Location Logging: verified that the system correctly records and transmits GPS data to Firebase in real time.

6.2.2 Sensor & Actuator Testing

- IMU (MPU6050): tested activity classification and impact detection for reliability and repeatability.
- Temperature (MLX90614): validated readings against a reference thermometer across the calibration dataset (see §5.2 / calibration notes).
- Buzzer: ensured the buzzer activates correctly on command and reliably cuts off at the 120-second safety limit.

6.2.3 System Performance Testing

- ESP32 Processing Speed: measured sensor read, classification, and Firebase push latency.
- Real-Time Data Handling: verified the ESP32 can handle simultaneous sensor inputs and send data to Firebase without lag.

6.2.4 Connectivity Testing

- WiFi connectivity behaviour under normal and degraded signal conditions were tested.
- Offline flash-cache validation: confirmed buffered readings are correctly queued during an outage and replayed once WiFi returns.

6.2.5 Power Supply Testing

- Voltage Stability: tested collar operation under full load using battery power.
- Battery Life: measured runtime between charges under typical usage.

6.2.6 Field Environment Testing

- On-Pet Wear Test: mounted the harness on the pet and evaluate comfort, fit, vibration resistance, and GPS/temperature/respiratory accuracy during real activity.
- Outdoor Testing: exposed the collar to varying temperatures, movement, and GPS conditions to test reliability in real-world use.

7. Source Code Repositories

Github repository url

<https://github.com/cepdnaclk/e21-3yp-PetGuard-Pro>

Github pages url

<https://cepdnaclk.github.io/e21-3yp-PetGuard-Pro/>

8. Appendix

8.1 Bill of Materials & Budget

Per-unit hardware cost breakdown (LKR):

Item(s)	Qty	Amount (LKR)
Jumper Wire + Solder Wire	5	730
ESP32 CP2102 Type-C	1	1,490.00
NEO-M8N GPS Module	1	3,290.00
SIM800L GPRS GSM Module	1	1,190.00
MLX90614 Contactless Temp Sensor	2	4,700.00
Accelerometer Module (MPU6050)	2	2220
5V Active Buzzer Alarm	2	130
Breadboard 830-Tie Point+Resistor Pack	2	760
MP1584 Ultra-Small Buck Converter	1	260
Soldering Paste Flux + project box	1	620
3.7V 1200mAh Li-Po Battery 802540	2	2,369.81
LED 5mm + connecting wire +SPST Rocker Switch	5	150
Vero Board Line Medium 14x6cm/Vero Board Dot Medium 14x6cm	2	160
Transistor + capacitor +Electrolytic Cap + Charger	9	141
TPS63020 Buck-Boost Converter	2	3,064.48
Padded Dog Harness +Enclosure +Velcro Strap	1	5,300.00
MAX30102 Oximeter/Heart Rate Sensor	1	350
		26 925.29

8.2 Key Dependencies

Component	Notable Dependencies
esp32-firmware	ArduinoJson, TinyGPSPlus, MPU6050, Adafruit MLX90614, Adafruit BusIO
MobileApp	firebase_core/auth/database/firestore/storage, flutter_riverpod, hive/hive_flutter, google_maps_flutter, geolocator, flutter_local_notifications, fl_chart, pdf
Admin_Dashboard	firebase, react 19, recharts, lucide-react, tailwindcss, vite, typescript